

IHK Algorithmen Handbuch 2025 – Detail

Java-ähnlicher Pseudocode (ohne Datentypen). Jede Sektion ist eine eigene Druckseite: Erklärung · Schritt-für-Schritt · Beispiel im IHK-Stil · Tipps · typische Fehler.

1) Zählen, Summieren, Durchschnitt

sehr häufig

Warum wichtig? Diese Aufgaben lassen sich schnell lösen und kommen in fast jeder AP2 vor. Sie prüfen, ob du Schleifen, Bedingungen und Rückgaben sicher beherrschst.

Kernidee

- *Zählen:* Zähler mit 0 starten, bei erfüllter Bedingung `count++`.
- *Summieren:* Akkumulator `sum` mit 0 starten, pro Element addieren.
- *Durchschnitt:* `sum / a.length` (nur wenn `a.length > 0`).

Pseudocode

```
funktion zaehleGroesser(a, grenze)
  count = 0
  for (i = 0; i < a.length; i++)
    if (a[i] > grenze) count++
  return count
```

```
funktion durchschnitt(a)
  if (a.length == 0) return 0 // falls leer nicht erwartet, sonst definieren
  sum = 0
  for (i = 0; i < a.length; i++) sum = sum + a[i]
  return sum / a.length
```

IHK-Stil Beispiel: `a={22, 27, 29, 23, 26}`, `grenze=25` → `count=3`; `sum=127`; `d=25,4`.

Tipp: Rückgabe immer *nach* der Schleife. Zähler/Akkumulator vorher initialisieren.

Fehler: `return` in der Schleife; Division durch 0 bei leerem Array; Ganzzahl-/Komma-Division in echten Sprachen.

2) Min / Max (1D-Array) sehr häufig

Warum wichtig? Absoluter Standard. Prüft Startwerte, Vergleichslogik und korrektes Schleifenmuster.

Kernidee

- Mit `a[0]` starten (nicht 0).
- Ab Index 1 iterieren und jeweils vergleichen.
- Bei Indexsuche zusätzlich `minIndex`/`maxIndex` führen.

Pseudocode

```
funktion findeMin(a)
  min = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] < min) min = a[i]
  return min
```

```
funktion findeMax(a)
  max = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] > max) max = a[i]
  return max
```

IHK-Stil Beispiel: `a={5,7,2,9,3}` → `min=2`, `max=9`. Schrittweise (Min): Start 5 → 7? nein → 2? ja → 9? nein → 3? nein.

Fehler: Startwert 0; return zu früh; `a.length` nicht geprüft.

3) Min / Max (2D) & Objektrückgabe

sehr häufig

Warum wichtig? 2D-Arrays testen dein Verständnis für verschachtelte Schleifen. Häufig in Aufgaben wie „Monatsverbrauch“.

Kernidee

1. Äußere Schleife: Zeilen/Monate; innere: Spalten/Tage.
2. Pro Monat summe bilden.
3. Nach dem Monat min/max vergleichen.
4. Objekt (z. B. Jahresstatistik) zurückgeben.

Pseudocode

```
funktion ermittleJahresstatistik(verbrauch) // Monate × Tage
    min = MAX_WERT; max = MIN_WERT
    for (m = 0; m < verbrauch.length; m++)
        summe = 0
        for (t = 0; t < verbrauch[m].length; t++)
            summe = summe + verbrauch[m][t]
        if (summe < min) min = summe
        if (summe > max) max = summe
    return new Jahresstatistik(min, max)
```

```
klasse Jahresstatistik(min, max)
// Attribute: minVerbrauch, maxVerbrauch
// Konstruktor & Getter
```

IHK-Stil Beispiel (Sommer 2022, stilisiert): 12×Tage → Monatssummen bilden → min/max bestimmen → return new Jahresstatistik(min,max). *Falle:* Tageswerte statt Monatssummen vergleichen.

Merke: Zwei Schleifen sind Pflicht; Objekt am Ende mit new zurückgeben.

4) If/Else + Counter (Aktie „besser“ als DAX)

häufig

Warum wichtig? Eine einfache Bedingung sauber anwenden und korrekt zählen – kommt oft in Börsen-/Qualitäts-/Grenzwertaufgaben.

Pseudocode

```
funktion notierungPlus(kurse)
  count = 0
  for (i = 0; i < kurse.length; i++)
    if (kurse[i].aktieProzent > kurse[i].daxProzent) count++
  return count
```

IHK-Stil Beispiel: (1.2,0.8), (-0.5,0.2), (0.3,0.1) → 2 Tage „besser“.

Fehler: $\geq$ statt $>$; Rückgabe in der Schleife; Zähler nicht initialisiert.

5) Lineare Suche

häufig

Warum wichtig? Must-Know. Entwickler-Alltag und Basis für viele Filteraufgaben.

Pseudocode

```
funktion linearSearch(a, x)
  for (i = 0; i < a.length; i++)
    if (a[i] == x) return i
  return -1
```

IHK-Stil Beispiel: a={4700, 4711, 4800}, x=4711 → Rückgabe 1; x=9999 → -1.

Tipp: Sobald gefunden → sofort return. Für Mehrfachtreffer ggf. zählen.

6) Binäre Suche (nur sortiert)

seltener

Warum wichtig? Effiziente Suche – manchmal als Bonus getestet. Unbedingt erwähnen: Array muss *sortiert* sein.

Pseudocode

```
funktion binarySearch(a, x)
  left = 0; right = a.length - 1
  while (left <= right)
    mid = (left + right) / 2
    if (a[mid] == x) return mid
    if (a[mid] < x) left = mid + 1
    else right = mid - 1
  return -1
```

IHK-Stil Beispiel: $a=\{2,4,7,9,11\}$, $x=7 \rightarrow \text{Mitte}=7 \rightarrow \text{Index } 2$. *Falle:* Unsortiertes Array.

7) BubbleSort

möglich

Warum wichtig? Einfaches Sortierverfahren, ideal, um Schleifen-Grenzen korrekt zu zeigen ($n-1-i$).

Pseudocode

```
funktion bubbleSort(a)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (a[j] > a[j+1]) swap(a[j], a[j+1])
```

IHK-Stil Beispiel: $a=\{5,2,4\} \rightarrow i=0: (5,2) \text{ swap} \rightarrow \{2,5,4\}; (5,4) \text{ swap} \rightarrow \{2,4,5\}; i=1: \text{ok} \rightarrow \text{Ergebnis } \{2,4,5\}.$

Merke: Innere Grenze $n-1-i$ spart Vergleiche; optional Abbruch, wenn keine Tausche in einer Runde.

8) InsertionSort

möglich

Warum wichtig? Stabil, gut für fast sortierte Daten. Demonstriert while-Schleife + Einfügeposition.

Pseudocode

```
funktion insertionSort(a)
  for (i = 1; i < a.length; i++)
    key = a[i]
    j = i - 1
    while (j >= 0 && a[j] > key)
      a[j+1] = a[j]
      j = j - 1
    a[j+1] = key
```

IHK-Stil Beispiel: $a = \{5, 2, 4\} \rightarrow i=1 \text{ key}=2$: 5 schieben $\rightarrow$ setze 2; $i=2 \text{ key}=4$: 5 schieben $\rightarrow$ setze 4 $\rightarrow \{2, 4, 5\}$.

Fehler: $a[j+1]=\text{key}$ vergessen; $j--$ vergessen $\rightarrow$ Endlosschleife.

9) Objektvergleich mit `vergleiche(x,y)`

oft bei Objekten

Warum wichtig? Bei Objekt-Arrays kann man nicht mit `>` vergleichen. Die IHK prüft, ob du eine Vergleichsfunktion korrekt einsetzt.

Pseudocode

```
funktion sort(a, vergleiche)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (vergleiche(a[j], a[j+1]) > 0) swap(a[j], a[j+1])
```

IHK-Stil Beispiel: Tageskurse nach Datum sortieren → `vergleiche(x,y)` gibt `>0` zurück, wenn x nach y steht.

Merke: Vergleichslogik gehört in die Funktion; der Sortiercode ruft nur `vergleiche` auf.

10) Strings & Prüfziffern (EAN/ISBN-13, Luhn)

sehr prüfungsnah

Warum wichtig? Prüft, ob du mit Zeichenketten, Positionen und Gewichtungen sicher umgehst – ein häufiger IHK-Klassiker.

EAN/ISBN-13

Erste 12 Ziffern mit Gewichten 1,3,1,3,... multiplizieren und summieren. Prüfziffer $p = (10 - (\text{Summe} \bmod 10)) \bmod 10$.

```
funktion pruefzifferEAN13(z12)
  sum = 0
  for (i = 0; i < 12; i++)
    z = z12[i]
    w = (i % 2 == 0) ? 1 : 3
    sum = sum + z * w
  p = (10 - (sum % 10)) % 10
  return p
```

Luhn

Von links mit Parität starten: jede zweite Ziffer verdoppeln; Ergebnisse $>9 \rightarrow -9$; $\text{Summe} \% 10 == 0 \rightarrow$ gültig.

```
funktion luhnGueltig(s)
  sum = 0; parity = s.length % 2
  for (i = 0; i < s.length; i++)
    d = s[i]
    if (i % 2 == parity)
      d = d * 2
      if (d > 9) d = d - 9
    sum = sum + d
  return (sum % 10) == 0
```

IHK-Stil Beispiel: $z12 = "400638133393"$ → Rechenschritte zeigen → p berechnen.

Fehler: falsche Startparität; Mod-10 ohne zweites $\% 10$.

11) Prüfungstipps & typische Fehler

- Variablen immer initialisieren (count=0, sum=0, min=a[0]).
- Bei 2D-Arrays innere Länge korrekt verwenden (m[r].length).
- Rückgaben am Ende; in der Schleife nur bei Suche.
- Objektrückgabe explizit mit new Klasse(...).
- Bei Vergleichen mit Objekten immer vergleiche nutzen.
- Klarer, sauberer Pseudocode zählt! Punkte gibt es für Struktur.

Druck: Browser → Drucken → Als PDF speichern. Jede Sektion hat einen Seitenumbruch.